MIT 8103 Advanced Database Systems: Technical Documentation

Continuous Assessment, September Cohort 2026

Student: Chukwuebuka Arueze | **Student ID:** 301912110 | **Email:** c.arueze2110@miva.edu.ng
Selected organisation: Benchokz Bank Limited (simulated retail bank / fintech platform)
GitHub repository: https://github.com/Benchokz/advanced-database-portfolio

1. Selected organisation and case study

For all five portfolios I used one made-up bank called **Benchokz Bank Limited**. It is a small retail bank that offers savings, current and wallet accounts to individuals, through its branches and a mobile app. Its database is an **OLTP** system (Online Transaction Processing), which means it handles many small, everyday actions such as deposits, withdrawals and transfers, and keeps a full record of each one for audit. Customers use it to check balances and send money. Tellers use it to open accounts and handle cash. Auditors use it to review history, and operations staff use it to reverse failed transactions. I kept the same bank in every portfolio so that each one builds on the one before it.

2. Portfolio 1: Database Design and Modelling

I designed six tables in PostgreSQL: *customers, branches, account_types, accounts, transactions* and *transfers*. I drew them as an ER diagram (entity-relationship diagram, a picture of the tables and how they link) on dbdiagram.io. I added rules so the database protects itself. Primary keys and foreign keys link the tables correctly, `UNIQUE` stops duplicates, and `CHECK` blocks bad values, for example a negative balance, an amount of zero, a customer who is too young, or a transfer from an account to itself. I then explained normalisation, which means splitting one big table into smaller ones so that each fact is stored only once, up to third normal form (3NF). I deliberately kept three values that could be calculated (`balance_after`, `accounts.balance` and `transfers.amount`) because they make the bank faster, and I wrote down the risk that they could get out of step. When I tested it, good data was accepted and bad data was refused by the database itself.

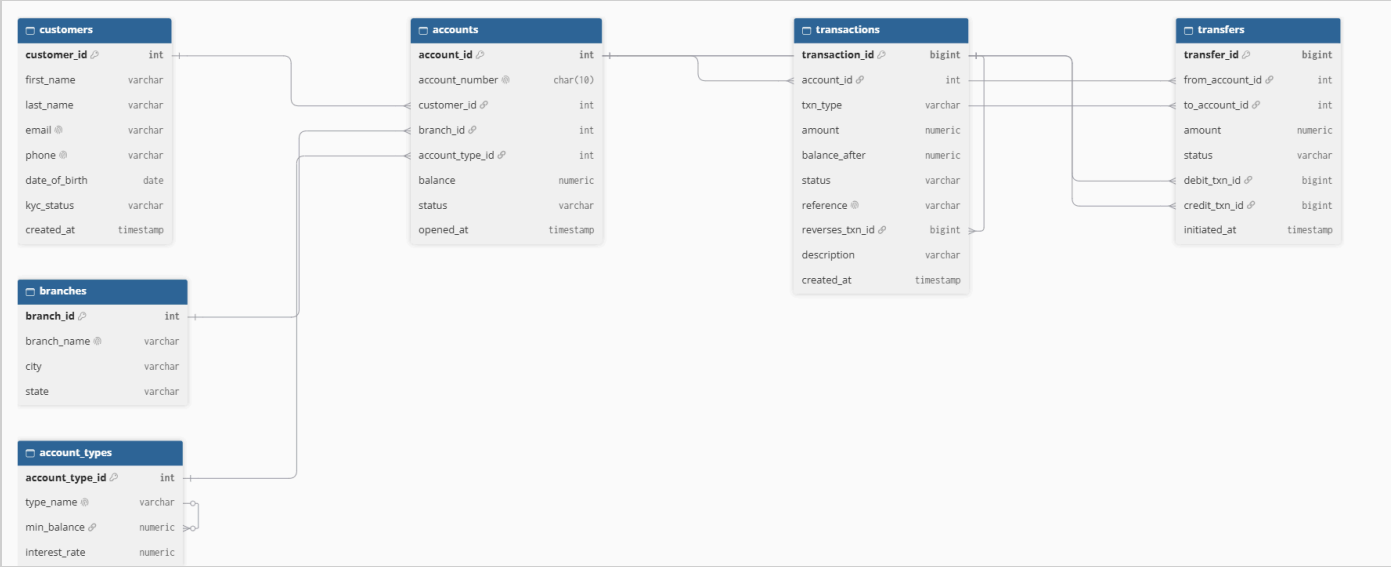


Figure 1: ER diagram of the six tables.

```
1  INSERT INTO accounts (account_number, customer_id, branch_id, account_type_id)
2  VALUES ('0000000001', 999, 1, 1);

ERROR:  insert or update on table "accounts" violates foreign key constraint "accounts_customer_id_fkey"
Key (customer_id)=(999) is not present in table "customers".

SQL state: 23503
Detail: Key (customer_id)=(999) is not present in table "customers".
```

Figure 2: An account for customer 999, who does not exist, is refused by the foreign key.

3. Portfolio 2: Query Processing and Optimisation

I filled the database with fake data, including 500,000 transactions, so that speed differences would be easy to see. I then ran five queries that a bank would really use, for example a customer's last 90 days of transactions and the number of failed transactions per day. I used `EXPLAIN ANALYZE`, which shows how PostgreSQL runs a query and how long it takes. After that I added six indexes. An index works like the index at the back of a book: it lets the database jump straight to the rows it needs instead of reading the whole table. I ran every test twice and used the second result, because the first run can be slower while data is read from disk.

Query	Before (ms)	After (ms)	How much faster
Q1 One account's statement	274.342 (read the whole table)	0.152 (used the index)	about 1,800 times
Q2 Top 10 customers	613.151	191.498	about 3 times (it still has to join tables)
Q3 A customer's accounts	4.108	0.091	about 45 times
Q4 Failed transactions per day	225.966	3.571 (index only)	about 63 times
Q5 Transfers for an account	22.735	0.230	about 99 times

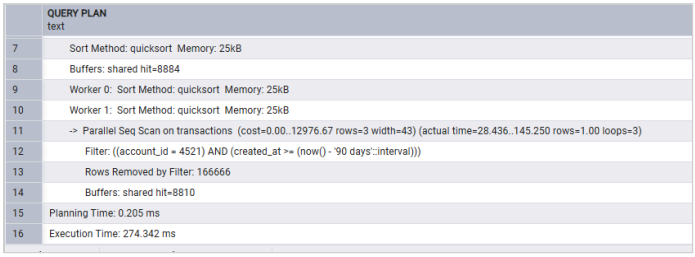


Figure 3a: Q1 before the index. It read the whole table and took 274.342 ms.

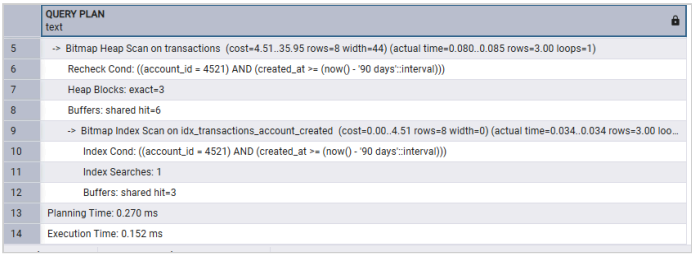


Figure 3b: Q1 after the index. It took 0.152 ms.

4. Portfolio 3: Transactions and Concurrency

A transaction groups several steps so that either all of them happen or none of them do. Using two pgAdmin windows (Session A and Session B) and two demo accounts, I showed a transfer that completed (`COMMIT`) and a failed transfer that was cancelled (`ROLLBACK`), where even the part that had already worked was undone. I also showed a transfer reversal that keeps the old records so auditors can follow the trail. Next I reproduced the **lost update** problem. Two customers withdrew 30,000 and 20,000 from a 100,000 account at the same time, and the balance ended at 80,000 when it should have been 50,000. I fixed it with `SELECT ... FOR UPDATE`, which locks the row so the second person waits for the first to finish, and the balance then ended correctly at 50,000. Finally I compared two isolation levels, `READ COMMITTED` and `REPEATABLE READ`, which control what one transaction can see of another's changes.

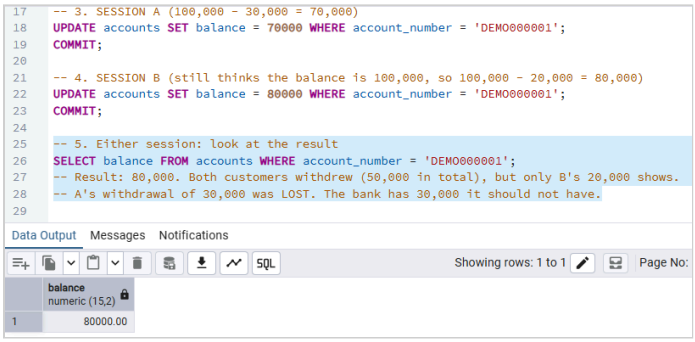


Figure 4a: Without locking, the balance ends at 80,000, which is wrong.

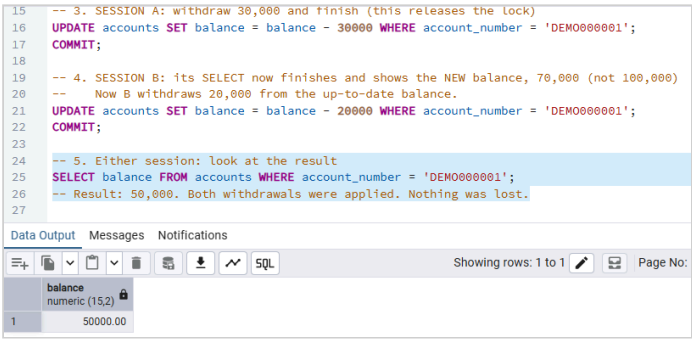


Figure 4b: With `FOR UPDATE`, the balance ends at 50,000, which is correct.

5. Portfolio 4: NoSQL and Advanced Data Models

NoSQL databases store data in more flexible ways than tables. I chose MongoDB, which stores **documents** (JSON-like records). I built two collections. *customers* keeps each customer's accounts inside the same document, because they are always read together. *transactions* is separate, because there can be thousands per customer, and each one has a `metadata` section that changes by channel (mobile app, ATM, POS, web or branch). I loaded 5,000 customers and 50,000 transactions and ran queries to find, insert, update and delete. I added a new USSD channel without changing any structure, which would have needed a table change in PostgreSQL. I also wrote five aggregation pipelines, which are MongoDB's way of grouping and joining data, and added indexes that changed the plan from scanning the whole collection (`COLLSCAN`) to using an index (`IXSCAN`). My conclusion is to use MongoDB for logs and customer profiles, and to keep money movement in PostgreSQL, so each database does the job it is best at.

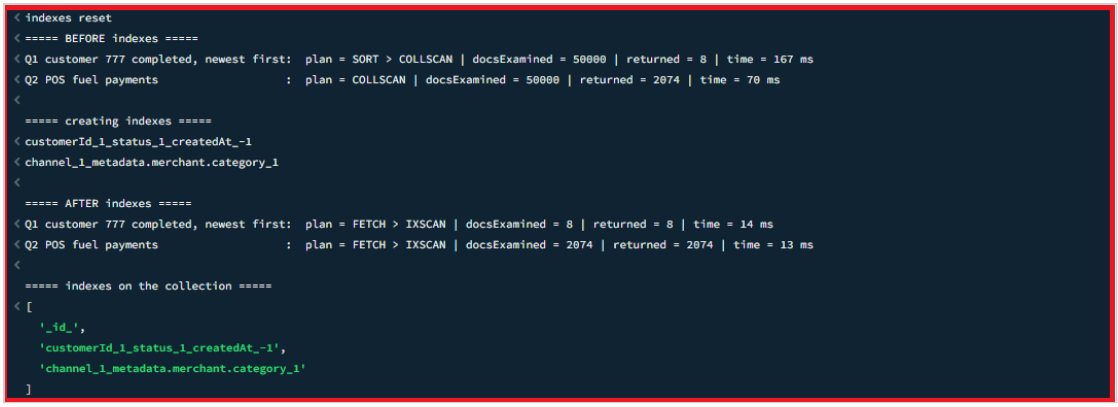


Figure 5: Before and after adding indexes. For Q1, documents read fell from 50,000 to 8 and the time from 167 ms to 14 ms.

6. Portfolio 5: Distributed and Cloud Database Exercise

A bank cannot stop working when one server fails, so I built a small **replica set**: three MongoDB servers (ports 27018, 27019 and 27020) that keep copies of the same data. One is the primary, which accepts writes, and the other two are secondaries. I wrote data once and read it back from a secondary, which proved it had been copied. Then I made the primary step down on purpose. Within seconds a different server, 27019, was elected as the new primary, no data was lost, and a new write still worked. I also compared **write concerns**, which decide how many servers must confirm a write before it counts as saved, and I explained why a bank should use `"majority"`. In writing I discussed the CAP trade-off (a distributed system cannot always give both perfect consistency and perfect availability when the network breaks), **sharding** (splitting a very large collection across servers, using `customerId` as the key), and running the database in the cloud. All three servers ran on my one laptop, so this shows how the mechanism works but gives no real protection against hardware failure. Sharding and cloud deployment are discussed but not built.

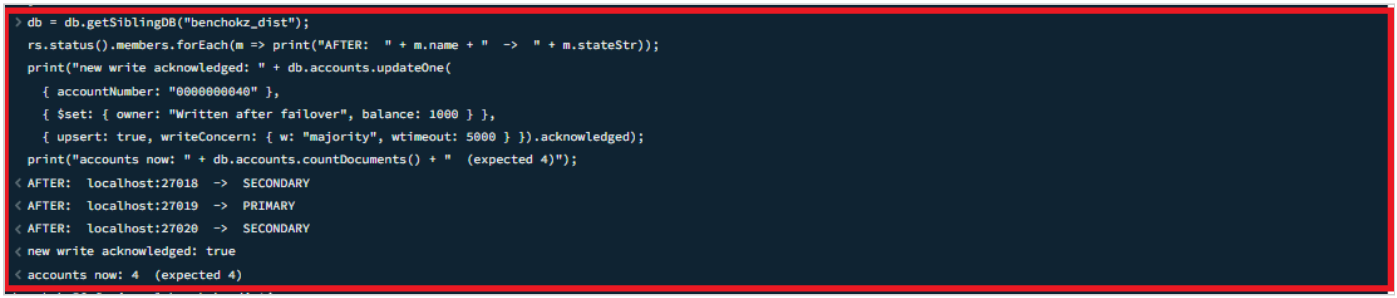


Figure 6: After the primary stepped down, 27019 became PRIMARY, the new write was accepted, and all 4 accounts were still there.

7. Individual technical reflection

Design choices. In PostgreSQL I split the data into six tables but kept a few calculated values for speed. In MongoDB I put accounts inside the customer document, but kept transactions separate because they keep growing. **Challenges.** My first timing tests looked slower than they really were because the data was being read from disk, so I repeated each test until the data was in memory to make the comparison fair. In Portfolio 5, my first failover test showed the old primary back in charge. It had a higher priority and its short step-down time had run out, so I repeated the test with a longer time and checked straight away. **What I learned.** An index helps most when a query needs only a few rows, and Q2 gained the least because of its joins. Locking prevents lost updates, but it makes people wait. Replication keeps the system available, while sharding helps it grow. No single database is best for everything, so for Benchokz Bank I would keep the money records in PostgreSQL and use MongoDB for logs and profiles.

8. AI use declaration

Tool used: Claude, an AI assistant by Anthropic. **What I used it for:** drafting the schema, scripts and README files for all five portfolios, planning the replica-set setup, explaining concepts to me, and drafting this document. **How I checked it:** I ran every script myself in pgAdmin, MongoDB Compass and PowerShell, took all the screenshots myself, repeated any test that gave an unexpected result, and checked that each explanation matches my own output. The reflection and conclusions are my own understanding.

9. References

- PostgreSQL Global Development Group. *PostgreSQL Documentation* (EXPLAIN, indexes, transaction isolation, explicit locking). https://www.postgresql.org/docs/
- MongoDB, Inc. *MongoDB Manual* (data modelling, aggregation, indexes, replication, sharding). https://www.mongodb.com/docs/manual/
- Elmasri, R. and Navathe, S. B. *Fundamentals of Database Systems* (normalisation and transactions).
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media (replication, partitioning, consistency).
- MIT 8103 Advanced Database Systems course materials; dbdiagram.io (ER diagram tool).